

Neural Machine Translation on IWSLT 2017

XU Yilin (Student ID: 60018664)
HAN Zhuo (Student ID: 60130387)
Course Code: CS5489

December 1, 2025

Abstract

We build a unified neural machine translation framework for English-German on the IWSLT 2017 dataset and use it to run a series of controlled experiments. Within this framework, we implement GRU, LSTM, and Transformer encoder-decoder models, and combine them with different tokenization strategies (whitespace, character-level, and sentencepiece), learning rate schedules, label smoothing strengths, vocabularies, and random seeds. We also extend our setup to a simple multilingual scenario by adding English-French data and language tags. We evaluate all systems using BLEU and chrF with both greedy and beam search decoding, and we further analyze the data and learned representations using visualization, PCA, and clustering. Overall, our study shows that a properly tuned Transformer with whitespace tokenization, moderate label smoothing, appropriate vocabulary size and a suitable learning rate schedule gives the strongest baseline, while GRU and LSTM remain solid and interpretable reference points. Beyond model performance, we report a systematic set of ablations and accompanying analyses to clarify how each design choice affects translation quality.

1 Introduction

In this project, we build a unified experimental framework for English-to-German translation on the IWSLT 2017 dataset. Within this framework, we implement and compare three model architectures: GRU with attention, LSTM with attention, and a Transformer encoder-decoder, under a range of tokenization and training configurations. Building on these baselines, we further explore several extensions commonly used in modern neural machine translation (NMT) systems, including label smoothing, Noam learning-rate scheduling with different warmup steps, and multilingual training with additional English-to-French data.

The project is driven by the following questions:

- **Model architecture:** How do GRU, LSTM, and Transformer models compare on a medium-scale translation task in terms of BLEU/chrF and training dynamics?
- **Tokenization:** How do different feature representations (character-level, whitespace, and sentencepiece subwords) affect translation quality and decoding behavior?
- **Training configuration:** How sensitive is the Transformer to label smoothing strength, learning-rate schedule, warmup steps, vocabulary size, and random seed?
- **Multilingual data:** Does adding English-to-French parallel data help or hurt the main English-to-German task?

To answer these questions, we perform a series of controlled experiments where we vary one factor at a time and evaluate each configuration using both greedy decoding and beam search,

reporting BLEU and chrF scores on the validation set. We also conduct data visualization and representation analysis, including sentence-length and token-frequency distributions and PCA-based visualization of learned word embeddings, complemented with K-means clustering.

Our contributions are threefold:

1. A unified NMT baseline framework that supports multiple architectures, tokenization schemes, decoding strategies, and training configurations, implemented in a reproducible way with clear logging and checkpointing.
2. A thorough empirical study on how architecture, tokenization, label smoothing, learning rate scheduling, vocabulary choices, random seeds, and multilingual training affect translation quality on IWSLT 2017.
3. Qualitative analysis of data and learned representations, using visualization and clustering to understand how the models organize linguistic information in the embedding space.

2 Task and Dataset

2.1 Task definition

The primary task in this project is supervised neural machine translation from English to German (En→De). Given a source sentence $x = (x_1, \dots, x_{T_x})$ in English, the goal is to generate a target sentence $y = (y_1, \dots, y_{T_y})$ in German that maximizes the conditional probability $p(y | x)$ under a parameterized sequence-to-sequence model.

During training, we minimize the cross-entropy loss between the predicted token distribution and the reference German translation, optionally with label smoothing. At inference time, we decode translations using either greedy decoding or beam search, both of which approximate the most likely target sequence under the trained model.

In the later part of the project, we extend this setup to a multilingual training scenario by adding English-to-French (En→Fr) data as an auxiliary task. We train on a mixture of En→De and En→Fr sentence pairs, and prepend a special language tag (e.g., <2de> or <2fr>) to the source sequence to indicate the desired target language. Evaluation remains focused on the main En→De task.

2.2 Dataset: IWSLT 2017

We conduct experiments on the IWSLT 2017 benchmark, using English-to-German (En→De) as the primary translation task. In later experiments, we additionally include English-to-French (En→Fr) parallel data as an auxiliary resource for a simple multilingual training setting, while keeping our main evaluation focused on En→De.

Table 1: Statistics of IWSLT2017 English–German (en→de) and English–French (en→fr) datasets.

Statistic	IWSLT2017 en→de	IWSLT2017 en→fr
Train size	206,112	232,825
Validation size	888	890
Test size	8,079	8,597
Average source length	17.31	17.37
Average target length	16.04	18.09
Source vocabulary size	127,478	136,220
Target vocabulary size	215,283	169,934

For each language pair, we use the official train/validation/test splits provided by the IWSLT 2017 dataset. To avoid data leakage, the validation set is kept strictly separate from training and is used only for model selection. Since K-fold cross-validation is impractical for NMT, we assess robustness by repeating key settings with multiple random seeds and summarizing performance across runs.

To characterize the scale and basic properties of the data, we report dataset statistics for each split and language pair in Table 1.

2.3 Preprocessing and tokenization

A key focus of this project is to compare different input representations for machine translation. In each experiment, we preprocess source and target sentences using one of the following tokenization strategies:

1. **Whitespace tokenization (word-level).** We split sentences on whitespace into word tokens. This representation is simple but can lead to out-of-vocabulary (OOV) issues, especially for rare words and morphologically rich forms.
2. **Character-level tokenization.** We decompose sentences into individual characters. This eliminates OOV tokens, but substantially increases sequence length and often makes optimization more difficult.
3. **Sentencepiece subword tokenization.** We train a sentencepiece model to obtain a subword vocabulary. Subword units reduce OOV while keeping sequence lengths more manageable than character-level tokenization.

For each tokenization scheme, we construct the corresponding vocabularies, add special tokens (e.g., `<pad>`, `<bos>`, `<eos>`), and convert sentences into sequences of token indices. In the multilingual setting, we prepend a language tag (e.g., `<2de>` or `<2fr>`) to the source sequence to indicate the desired target language. Across all settings, we keep the rest of the data pipeline consistent so that tokenization can be treated as an isolated experimental factor.

3 Methods

3.1 Model architectures

We implement three sequence-to-sequence models in a unified codebase to enable fair comparisons under the same training and decoding pipeline.

GRU with attention. Our first model uses a GRU encoder and a GRU decoder with additive attention. The encoder reads the source sequence and produces a sequence of hidden states. At each decoding step, the decoder attends over these encoder states to compute a context vector, which is combined with the decoder state to predict the next target token. Unless otherwise noted, we use an embedding size of 256, a hidden size of 512, 2 GRU layers, and dropout 0.2 in both the encoder and decoder.

LSTM with attention. The second model follows the same overall structure as the GRU system, but replaces GRUs with LSTMs. We keep the embedding size, hidden size, number of layers, and dropout fixed, so the main difference is the recurrent cell type. This isolates the effect of using LSTM versus GRU for this medium-scale translation setting.

Transformer encoder-decoder. The third model is a standard Transformer encoder-decoder. We use token embeddings with sinusoidal positional encodings, multi-head self-attention in both encoder and decoder, and encoder-decoder attention in the decoder. Unless otherwise noted, we use a model dimension of 256, a feed-forward dimension of 1024, 4 encoder layers, 4 decoder layers, and 4 attention heads, with dropout 0.2. We follow the original design with residual connections and layer normalization.

All three models share a common interface: they take token-id sequences as input and output a distribution over the next target token at each time step. This design allows us to reuse the same training loop, loss function, and decoding implementations across architectures.

3.2 Tokenization strategies

We treat tokenization as a controllable feature representation and support three options: whitespace tokenization, character-level tokenization, and sentencepiece subword tokenization, as described in Section 2.3. For each option, we build a separate vocabulary with special tokens `<pad>`, `<bos>`, and `<eos>`. In the multilingual experiments, we also add language tags like `<2de>` and `<2fr>` as extra tokens and prepend them to the source sentence to tell the model which language to generate.

3.3 Training setup

Loss and label smoothing. We optimize the cross-entropy loss over target tokens. We optionally apply label smoothing with smoothing factors in $\{0.0, 0.1, 0.2, 0.4\}$. When label smoothing is enabled, we replace the one-hot target distribution with a softened distribution, which is known to reduce overconfidence and sometimes improve generalization.

Optimizer and learning rate schedules. We use the Adam optimizer for all models. On top of Adam, we experiment with two learning rate options:

- **ReduceLROnPlateau:** the learning rate is reduced when the validation loss stops improving for several epochs,
- **Noam scheduler:** a Transformer-style schedule where the learning rate first increases linearly during a warmup period and then decays proportionally to the inverse square root of the step number. We vary the number of warmup steps and study its effect on the Transformer performance.

Mixed precision and gradient clipping. To speed up training and save GPU memory, we optionally use mixed-precision training with `torch.amp.autocast` and `GradScaler`. We also apply gradient clipping with a maximum global norm of 1.0 for all models to stabilize training.

Early stopping and checkpointing. We monitor the validation loss at the end of each epoch. If the validation loss does not improve beyond a small margin (`min_delta`) for a fixed number of epochs (`early_stop_patience`), we stop training early. We always save the model checkpoint that achieves the best validation loss, together with the optimizer, learning rate scheduler, AMP scaler, and all configuration arguments.

3.4 Decoding: greedy vs beam search

At test time, we use two decoding strategies:

- **Greedy decoding:** at each time step we pick the token with the highest probability. This is simple and fast, but can miss better sequences.

- **Beam search:** we maintain a beam of the top- B partial hypotheses and expand them until all beams produce an `<eos>` token or reach a maximum length. Beam search is slower but usually gives better BLEU scores.

For every experiment, we report both greedy and beam search performance (BLEU and chrF), which also lets us see how much each model benefits from a more expensive decoding strategy.

3.5 Multilingual extension

To study the effect of additional parallel data, we extend our setup to a simple multilingual training setting that includes both English-German and English-French sentence pairs. During training, we mix En→De and En→Fr examples (including within the same mini-batches) and prepend a language tag to the source sentence (e.g., `<2de>` or `<2fr>`) to indicate the desired target language. At evaluation time, we report results only on En→De. This design tests whether adding French as an auxiliary language helps or hurts the main English-to-German task under a fixed model capacity.

4 Experimental setup

4.1 Train/validation split and tuning protocol

We use the standard train/validation split provided by the IWSLT 2017 dataset. All models are trained on the training set, and we use the validation set to: (1) choose hyperparameters such as label smoothing strength and learning rate schedule, (2) decide when to stop training (early stopping), and (3) select the best checkpoint.

We do not perform full K-fold cross-validation, because for NMT this would be very expensive in terms of time and compute. Instead, we try to reduce variance in two ways: (1) for key settings, we repeat some experiments with different random seeds and compare the spread of BLEU and chrF scores; (2) we report both BLEU and chrF on the same validation set, so our conclusions do not rely on a single metric.

4.2 Evaluation metrics

We use two standard metrics:

- **BLEU:** computed with SacreBLEU, which normalizes tokenization and allows fair comparison across runs. BLEU measures n-gram overlap between our translations and the reference translations.
- **chrF:** also from SacreBLEU, based on character n-gram F-scores. chrF is often more sensitive to small word-level variations and is less affected by exact tokenization.

For each experiment, we report BLEU and chrF on the validation set, for both greedy decoding and beam search.

4.3 Implementation details

All experiments are implemented in PyTorch. We use mini-batch training with shuffled batches and fix reasonable maximum sequence lengths to avoid extremely long sentences. We log training and validation losses, current learning rate, and training/evaluation time for each epoch. We also save loss curves and metrics in JSON format so that we can later visualize and compare different runs.

Table 2: Main results on IWSLT2017 En→De under different architectures and tokenization strategies. We report sacreBLEU and chrF for greedy decoding and beam search.

Tokenizer	Model	Greedy		Beam	
		sacreBLEU	chrF	sacreBLEU	chrF
Whitespace	GRU	23.85	42.41	26.57	45.54
	LSTM	24.65	42.95	27.91	46.10
	TFM	26.37	51.08	28.98	52.87
Sentencepiece	GRU	12.61	39.58	14.45	42.41
	LSTM	15.74	43.70	18.27	46.96
	TFM	18.78	49.51	21.08	51.20
Character	GRU	21.50	48.61	24.90	51.39
	LSTM	23.04	50.22	25.48	52.37
	TFM	19.35	47.80	22.12	48.94

5 Results and analysis

In this section we summarize our main empirical findings on ISWLT 2017 EN→De. Main results are shown in Table 2.

5.1 GRU vs LSTM vs Transformer

We begin by comparing the three model families, GRU with attention, LSTM with attention, and the Transformer, under the same evaluation protocol on En→De (Table 2). Overall, the Transformer provides the strongest results among the three architectures, while the two RNN-based models form a competitive second tier.

Across the settings reported in Table 2, the Transformer achieves the highest scores in most cases. For example, under whitespace tokenization it reaches 28.98 sacreBLEU and 52.87 chrF with beam search, outperforming the LSTM (27.91/46.10) and GRU (26.57/45.54). Under sentencepiece tokenization, the Transformer remains the top model (21.08/51.20 with beam), again ahead of LSTM (18.27/46.96) and GRU (14.45/42.41).

Comparing the two recurrent architectures, LSTM is consistently stronger than GRU across the reported runs, although the margin is generally moderate. This suggests that, within our implementation and training setup, replacing GRU with LSTM yields a reliable but not dramatic improvement, while the Transformer offers the best overall performance when properly trained.

5.2 Effect of tokenization

We next examine how tokenization affects translation quality by comparing whitespace, character-level, and sentencepiece tokenization for each model architecture (Table 2). Overall, whitespace tokenization gives the strongest results in our current setup. For all three architectures, whitespace yields the best beam sacreBLEU, and it is also competitive in chrF. The best overall scores in Table 2 are achieved by the whitespace Transformer, which reaches 28.98 sacreBLEU and 52.87 chrF with beam search.

Character-level tokenization is generally competitive, particularly for the recurrent models. Both GRU and LSTM achieve higher chrF with character-level inputs than with whitespace, and LSTM reaches 25.48 sacreBLEU and 52.37 chrF with beam search. However, for the Transformer,

Table 3: Effect of label smoothing on the Transformer for IWSLT2017 En→De. We report sacreBLEU and chrF under greedy decoding and beam search.

Tokenizer	ϵ	Greedy		Beam	
		sacreBLEU	chrF	sacreBLEU	chrF
Whitespace	0.0	26.37	51.08	28.98	52.87
	0.1	25.78	51.69	29.86	53.65
	0.2	27.01	51.72	31.16	53.59
	0.4	26.78	52.03	30.74	53.86
Sentencepiece	0.0	18.78	49.51	21.08	51.20
	0.1	20.21	50.64	21.97	52.09
	0.2	20.76	51.06	22.45	52.55
	0.4	21.21	50.59	22.93	52.31
Character	0.0	19.35	47.80	22.12	48.94
	0.1	19.34	47.08	22.22	48.44
	0.2	18.38	47.28	21.38	47.60
	0.4	16.33	45.84	18.71	46.21

character-level tokenization is less favorable, with a noticeable drop compared to whitespace in both BLEU and chrF.

In contrast, sentencepiece tokenization underperforms in these experiments. Across all three architectures, sentencepiece yields substantially lower BLEU than the corresponding whitespace and character-level settings. This suggests that the sentencepiece configuration in our pipeline is not yet well tuned, and its benefits are not realized under the current training and vocabulary choices.

5.3 Effect of label smoothing

We next study label smoothing for the Transformer by varying the smoothing factor $\epsilon \in \{0.0, 0.1, 0.2, 0.4\}$ under different tokenization schemes (Table 3). The effect is clearly dependent on the tokenizer. With whitespace tokenization, moderate smoothing substantially improves beam search BLEU, increasing from 28.98 at $\epsilon = 0.0$ to 31.16 at $\epsilon = 0.2$, while chrF remains consistently high. With sentencepiece tokenization, label smoothing also improves over the no smoothing baseline, with beam BLEU rising from 21.08 to the 22.45 to 22.93 range when $\epsilon \geq 0.2$.

In contrast, character-level tokenization does not benefit from label smoothing in our setup. Performance is roughly unchanged at very small smoothing, but degrades as ϵ increases, with a clear drop at $\epsilon = 0.4$ for both BLEU and chrF. Overall, these results suggest that moderate label smoothing is helpful for whitespace and sentencepiece tokenization, but should be avoided for character-level inputs in our Transformer pipeline.

5.4 Effect of learning rate scheduler and warmup

We focus on the Transformer with whitespace tokenization and no label smoothing, and compare two learning rate strategies: our default ReduceLROnPlateau scheduler and the Noam schedule with different warmup steps (Table 4). Overall, switching from ReduceLROnPlateau to Noam yields a small improvement in BLEU under beam search, with the best setting reaching 29.60 sacreBLEU compared to 28.98 for the default scheduler. The chrF scores are relatively stable across these settings, suggesting that the gains mainly appear in BLEU.

Table 4: Learning rate scheduling comparison for the Transformer on IWSLT2017 En→De with whitespace tokenization and label smoothing $\epsilon = 0.0$. ReduceLROnPlateau is the default setting. Noam results are shown with different warmup steps.

Scheduler	Warmup steps	Greedy		Beam	
		sacreBLEU	chrF	sacreBLEU	chrF
ReduceLROnPlateau	–	26.37	51.08	28.98	52.87
Noam	1000	27.20	50.86	29.42	52.36
	1500	26.67	50.97	29.14	52.60
	2500	27.49	50.97	29.60	52.38

Table 5: Effect of vocabulary size for the Transformer with whitespace tokenization on IWSLT2017 En→De.

Vocab size	Greedy		Beam	
	sacreBLEU	chrF	sacreBLEU	chrF
8K	29.58	53.22	34.64	55.33
16K	27.19	52.29	30.03	53.70
20K	26.37	51.08	28.98	52.87

Within the Noam schedule, the choice of warmup steps matters. Increasing warmup from 1000 to 2500 slightly improves beam BLEU, although the trend is not strictly monotonic and the differences are modest. In our experiments, warmup mainly acts as a fine-tuning knob rather than a dominant factor.

5.5 Effect of vocabulary and random seed

We next examine how sensitive the whitespace Transformer is to vocabulary size and random seed (Tables 5 and 6). Vocabulary size has a large impact in our experiments. Reducing the vocabulary from 20K to 8K yields a substantial improvement, with beam sacreBLEU increasing from 28.98 to 34.64 and chrF increasing from 52.87 to 55.33. A mid-sized vocabulary of 16K performs in between, suggesting that the choice of vocabulary size is an important tuning factor for this dataset and model configuration.

We also observe non-trivial variance across random seeds even when the configuration is fixed. In our runs, beam sacreBLEU ranges from 24.49 to 28.98 across four seeds, and chrF varies from 51.82 to 52.87. This spread indicates that reporting a single run can be misleading, and that comparisons between settings should ideally account for seed variance.

5.6 Effect of adding French data

Finally, we evaluate a multilingual extension that jointly trains on En→De and En→Fr with language tags, while reporting En→De performance only (Table 7). Overall, adding French data is beneficial in our experiments, but the magnitude and consistency of the gains depend on the tokenizer and label smoothing.

With whitespace tokenization, multilingual training leads to clear improvements over the corresponding monolingual baselines. For example, with label smoothing $\epsilon = 0.2$, beam sacreBLEU

Table 6: Effect of random seed for the Transformer with whitespace tokenization on IWSLT2017 En→De.

Seed	Greedy		Beam	
	sacreBLEU	chrF	sacreBLEU	chrF
0	25.03	50.57	28.65	52.58
42	26.37	51.08	28.98	52.87
128	25.81	50.59	28.86	52.61
512	22.07	49.12	24.49	51.82

Table 7: Multilingual training with En→De and En→Fr, evaluated on En→De. We compare multilingual training (Multi) with the corresponding monolingual baseline (Mono) under different tokenizers and label smoothing values.

Tokenizer	Setting (LS ϵ)	Greedy		Beam	
		sacreBLEU	chrF	sacreBLEU	chrF
Whitespace	Multi (0.0)	26.87	51.67	30.44	54.78
	Multi (0.2)	29.45	54.66	33.95	56.70
	Mono (0.0)	26.37	51.08	28.98	52.87
	Mono (0.2)	27.01	51.72	31.16	53.59
Sentencepiece	Multi (0.0)	20.50	51.35	22.52	53.05
	Multi (0.2)	20.11	51.02	22.27	52.49
	Mono (0.0)	18.78	49.51	21.08	51.20
	Mono (0.2)	20.76	51.06	22.45	52.55
Character	Multi (0.0)	20.58	48.80	23.56	49.74
	Multi (0.2)	21.05	48.61	23.59	49.46
	Mono (0.0)	19.35	47.80	22.12	48.94
	Mono (0.2)	18.38	47.28	21.38	47.60

increases from 31.16 in the monolingual setting to 33.95 in the multilingual setting, and chrF increases from 53.59 to 56.70. Even without label smoothing, multilingual training improves beam sacreBLEU from 28.98 to 30.44 and raises chrF from 52.87 to 54.78. These results suggest that additional parallel data can substantially strengthen the whitespace Transformer in our fixed-capacity setup.

For sentencepiece, multilingual training improves the no-smoothing baseline (beam sacreBLEU 21.08 to 22.52), but does not improve over the best monolingual setting with label smoothing. Character-level tokenization also benefits from multilingual training, especially compared to the monolingual runs where label smoothing is harmful. Overall, the strongest multilingual result in our experiments is achieved with whitespace tokenization and $\epsilon = 0.2$.

6 Data visualization and representation analysis

Besides reporting scores, we also look at the data and the learned representations using our visualization notebook.

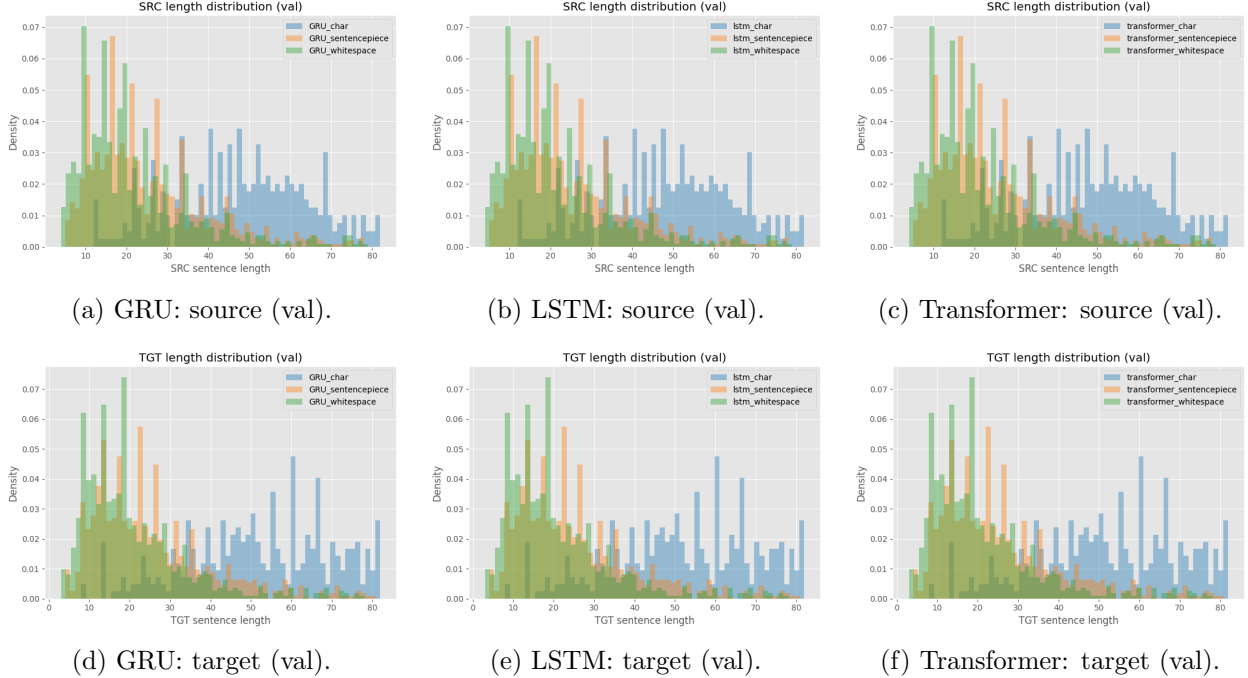


Figure 1: Sentence length distributions on the validation set after tokenization. Top row shows source lengths and bottom row shows target lengths for GRU, LSTM, and Transformer. Across all models, character-level tokenization produces substantially longer sequences than whitespace or sentencepiece tokenization, while sentencepiece tends to slightly shorten sequences compared to whitespace tokenization due to shared subword segmentation.

6.1 Sentence length and token frequency distributions

We plot the distribution of source and target sentence lengths (after tokenization) under three tokenizers (character-level, whitespace, and sentencepiece) for all three model setups (GRU, LSTM, and Transformer). As shown in Figure 1, character-level tokenization consistently produces much longer sequences than whitespace or sentencepiece tokenization on both the source and target sides. In contrast, whitespace and sentencepiece concentrate on shorter lengths with similar overall shapes, while sentencepiece is typically slightly more compact than pure whitespace tokenization due to its use of reusable subword units.

We also compute token frequency statistics and visualize the top- K most frequent target-side tokens under the *whitespace* tokenizer. Figure 2 shows the top-50 tokens (excluding special tokens)

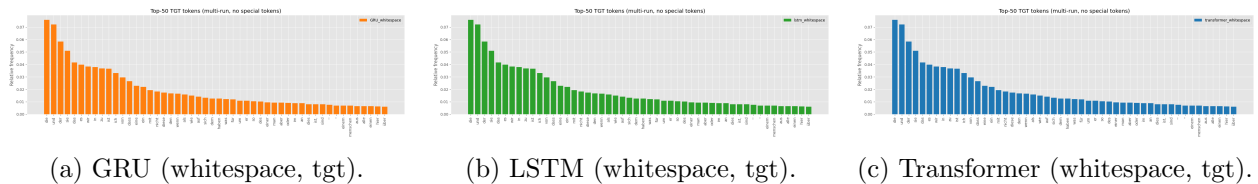


Figure 2: Top-50 most frequent target-side tokens (relative frequency) under whitespace tokenization, excluding special tokens. A small number of highly frequent tokens accounts for a large fraction of occurrences, followed by a rapid drop-off, exhibiting a typical long-tail distribution that is consistent across GRU, LSTM, and Transformer.

for GRU, LSTM, and Transformer. Across all three models, the distribution is strongly skewed: a small set of very frequent function words dominates the token mass, while the remaining tokens quickly drop in relative frequency, reflecting a clear long-tail pattern. The ranked profiles are highly similar across architectures, suggesting that the observed token-frequency structure is largely driven by the dataset and tokenization scheme rather than the specific model family.

6.2 Embedding space analysis with PCA

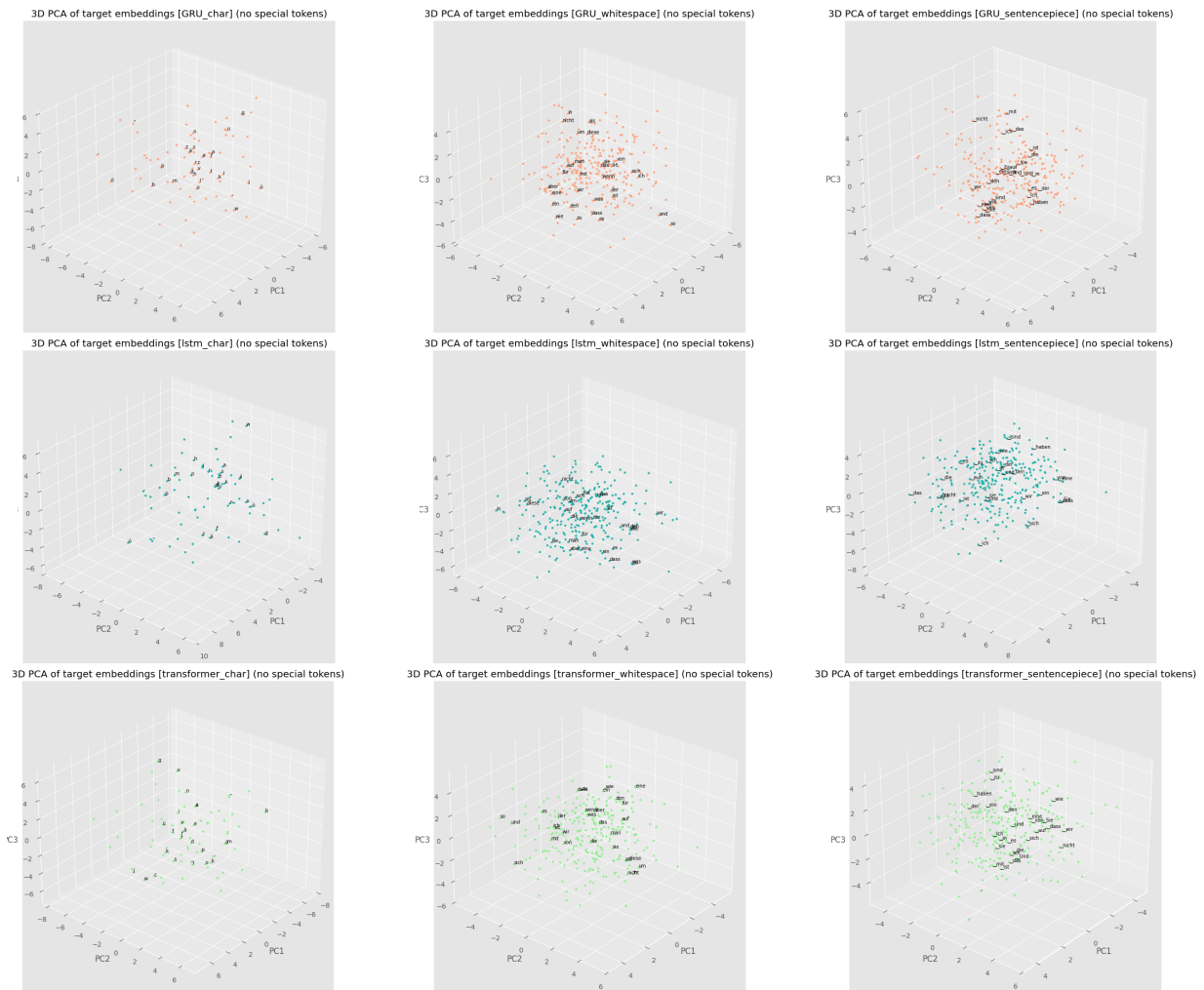


Figure 3: 3D PCA visualization (PC1-PC3) of target embedding vectors (special tokens removed), with a subset of frequent tokens annotated. Rows correspond to model families (GRU/LSTM/Transformer) and columns correspond to tokenizers (char/whitespace/sentencepiece).

We next inspect the geometry of the learned *target-side* embedding space by applying PCA to the target embedding matrix (special tokens removed) and visualizing the first three principal components (PC1-PC3), with a subset of frequent tokens annotated (Figure 3). Across all settings, the embeddings form a dense, cohesive cloud with only a small number of clearly separated points,

indicating that most tokens share a common high-variance subspace in the top PCs rather than splitting into sharply separated groups.

Tokenization leads to the most visible qualitative differences. Under character-level tokenization, the plotted points correspond to individual characters and thus appear comparatively sparse, with weaker local concentration and fewer visually dense regions. In contrast, whitespace and sentencepiece tokenization produce much denser point clouds: frequently occurring tokens tend to concentrate near the central high-density region, while a handful of points lie farther away as outliers. The overall shape and spread across GRU, LSTM, and Transformer are broadly similar under the same tokenizer, suggesting that the coarse PCA geometry is primarily driven by the token inventory and frequency statistics induced by the tokenizer, with architecture-dependent effects being relatively subtle at this level of projection.

7 Discussion

This project is guided by four questions: (i) how model architecture influences translation quality and training dynamics, (ii) how tokenization affects quality and decoding behavior, (iii) how sensitive the Transformer is to key training configurations, and (iv) whether adding auxiliary English–French data changes English–German performance. We organize the discussion accordingly.

7.1 Model architecture: GRU vs LSTM vs Transformer

Across our experiments on IWSLT2017 En→De, the Transformer achieves the strongest overall translation performance when paired with a suitable tokenization and training configuration, while recurrent baselines provide competitive but generally lower results. In terms of quality, the Transformer is typically best under whitespace tokenization, especially when evaluated with beam search, whereas GRU and LSTM tend to be closer to each other with LSTM usually offering a modest improvement. In terms of training behavior, recurrent models often train more smoothly under character-level tokenization (where sequences are long but the vocabulary is small), while the Transformer is more sensitive to design choices such as label smoothing and vocabulary size. Overall, if the goal is to maximize BLEU/chrF on this medium-scale task, the Transformer is the most effective architecture in our setting, with recurrent models serving as simpler baselines that are easier to stabilize under some tokenizations.

7.2 Tokenization: character-level vs whitespace vs sentencepiece

Tokenization strongly shapes both the difficulty of the learning problem and the behavior of decoding. The sentence-length visualizations show that character-level tokenization greatly increases sequence lengths on both source and target sides, which increases decoding steps and expands the effective context the model must handle. This can hurt the Transformer more noticeably, while the recurrent models may remain relatively competitive in chrF under character tokenization.

Whitespace tokenization is the most reliable choice in our pipeline: it consistently supports strong performance across architectures and yields the best overall setting when combined with the Transformer. Sentencepiece does not outperform whitespace in our experiments, indicating that subword methods are not automatically beneficial without careful tuning. This mismatch is consistent with the idea that sentencepiece performance depends on choices such as vocabulary size and training parameters, and may also interact with learning-rate schedules and regularization.

From a decoding perspective, beam search improves results for all tokenizers, but the gain is most pronounced in the stronger whitespace-based systems, suggesting that better token-level

modeling (e.g., common function words and frequent patterns) translates more efficiently into search-time improvements.

7.3 Training configuration: sensitivity of the Transformer

We find that the Transformer is measurably sensitive to multiple training knobs.

Label smoothing. Moderate label smoothing improves the whitespace Transformer, especially under beam decoding, while overly strong smoothing can degrade performance, and the optimal value differs across tokenizers. This suggests that label smoothing acts as a regularizer whose effective strength depends on the token inventory and sequence characteristics induced by the tokenizer.

Learning-rate schedule and warmup. Switching from a plateau-based scheduler to a Noam-style schedule yields modest but consistent improvements in beam BLEU for the Transformer, while chrF changes are smaller. Warmup steps also matter, but primarily as a stability and fine-tuning factor rather than the dominant driver of performance.

Vocabulary size. Vocabulary size produces one of the clearest effects in our study: under whitespace tokenization, an appropriately reduced vocabulary significantly improves performance, indicating that the default vocabulary choice can be suboptimal on IWSLT-scale data. This also provides a plausible explanation for why sentencepiece underperforms here: the subword vocabulary and segmentation may not be well matched to our training setup.

Random seed. Seed variance is non-negligible even under fixed settings, which means that small score differences should be interpreted cautiously. Reporting either multiple seeds or at least validating that improvements exceed typical seed variation is important for reliable conclusions.

7.4 Multilingual data: adding English–French auxiliary data

We further test whether adding English–French parallel data as an auxiliary task changes the main English–German performance under joint training with language tags. In our results, auxiliary data can be beneficial, but the effect depends on the tokenization and base configuration: improvements are clearest when the underlying Transformer setup is already strong (notably whitespace with suitable regularization). This suggests that the auxiliary data acts more as a data augmentation or representation-sharing benefit rather than a universal fix for weak configurations. At the same time, we do not observe catastrophic degradation in the main task under the tag-based setup, indicating that the joint training approach is feasible and can yield small gains when calibrated properly.

7.5 Limitations and implications

Our findings are specific to IWSLT2017 En→De and the particular hyperparameter ranges explored. In particular, sentencepiece appears under-tuned relative to whitespace in our implementation, so its weaker performance should be interpreted as a result about this pipeline rather than a general statement about subword tokenization. More broadly, the study suggests a practical workflow for medium-scale NMT: start from a whitespace Transformer baseline, tune vocabulary size and label

smoothing, confirm gains under beam decoding, and only then expand to auxiliary multilingual training.

8 Conclusion

We implemented a unified neural machine translation framework for IWSLT 2017 En→De and used it to run a systematic set of controlled experiments across model architectures (GRU, LSTM, Transformer), tokenization strategies (whitespace, character-level, sentencepiece), training configurations (label smoothing, learning-rate schedules, warmup steps, vocabulary sizes, random seeds), and decoding strategies (greedy vs beam search). Overall, the strongest and most stable baseline in our experiments is obtained by a properly tuned Transformer under whitespace tokenization, with additional gains achievable through moderate label smoothing, appropriate scheduling, and especially an improved choice of vocabulary size. We also find that seed variance is non-trivial and that adding auxiliary parallel data can further improve En→De performance under the tagged joint-training setup.

In addition to reporting BLEU and chrF, we conducted visualization and representation analyses to better understand the impact of tokenization on sentence lengths, token frequency structure, and the geometry of learned target embeddings. Taken together, these results provide a clearer picture of which design choices matter most for building strong NMT baselines on this medium-scale dataset, and they leave a concrete set of follow-up directions for strengthening the weaker configurations (notably sentencepiece) in future work.